

**LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 2**



ANDROID LAYOUT WITH COMPOSE

Oleh:

Nazmi Hakim NIM. 2310817210012

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
APRIL 2025**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE
MODUL 2

Laporan Praktikum Pemrograman Mobile Modul 2: Android Layout With Compose ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Nazmi Hakim
NIM : 2310817210012

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Raka Azwar
NIM. 2210817210012

Andreyan Rizky Baskara, S.Kom., M.Kom.
NIP. 19930703 201903 01 011

DAFTAR ISI

LEMBAR PENGESAHAN	2
DAFTAR ISI	3
DAFTAR GAMBAR.....	4
DAFTAR TABEL	5
SOAL 1.....	6
A. Source Code.....	7
B. Output Program	14
C. Pembahasan	19
D. Tautan Git	22
SOAL 2.....	23
A. Pembahasan	23

DAFTAR GAMBAR

Gambar 1. Tampilan Awal Aplikasi .	6
Gambar 2. Tampilan Pilihan Persentase Tip	7
Gambar 3. Screenshot Hasil Jawaban Soal 1 XML	14
Gambar 4. Screenshot Hasil Jawaban Soal 1 XML	15
Gambar 5. Screenshot Hasil Jawaban Soal 1 XML	16
Gambar 6. Screenshot Hasil Jawaban Soal 1 Jetpack Compose	17
Gambar 7. Screenshot Hasil Jawaban Soal 1 Jetpack Compose	18
Gambar 8. Screenshot Hasil Jawaban Soal 1 Jetpack Compose	19

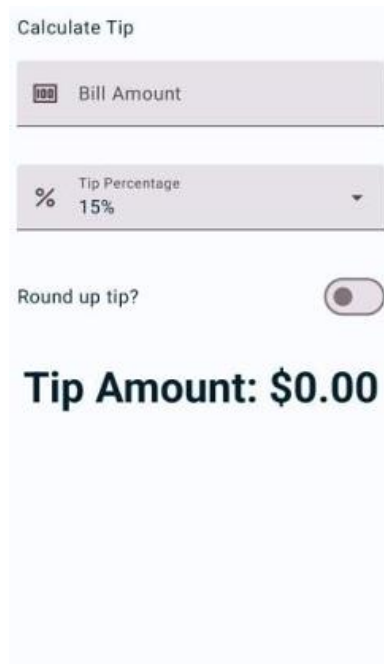
DAFTAR TABEL

Tabel 1. Source Code Jawaban Soal 1 XML	7
Tabel 2. Source Code Jawaban Soal 1 XML	9
Tabel 3. Source Code Jawaban Soal 1 XML	11
Tabel 3. Source Code Jawaban Soal 1 Jetpack Compose	11

SOAL 1

Buatlah sebuah aplikasi kalkulator tip menggunakan XML dan Jetpack Compose yang dirancang untuk membantu pengguna menghitung tip yang sesuai berdasarkan total biaya layanan yang mereka terima. Fitur-fitur yang diharapkan dalam aplikasi ini mencakup:

- Input biaya layanan: Pengguna dapat memasukkan total biaya layanan yang diterima dalam bentuk nominal.
- Pilihan persentase tip: Pengguna dapat memilih persentase tip yang diinginkan.
- Pengaturan pembulatan tip: Pengguna dapat memilih untuk membulatkan tip ke angka yang lebih tinggi.
- Tampilan hasil: Aplikasi akan menampilkan jumlah tip yang harus dibayar secara langsung setelah pengguna memberikan input.



Gambar 1. Tampilan Awal Aplikasi .



Gambar 2. Tampilan Pilihan Persentase Tip

A. Source Code

XML :

activity_main.xml :

Tabel 1. Source Code Jawaban Soal 1 XML

1	<?xml	version="1.0"	encoding="utf-8"?>
2	<LinearLayout		
3	xmlns:android="http://schemas.android.com/apk/res/android"		
4	android:layout_width="match_parent"		
5	android:layout_height="match_parent"		
6	android:orientation="vertical"		
7	android:padding="24dp">		
8			
9	<TextView		
10	android:id="@+id/titleText"		
11	android:layout_width="wrap_content"		
12	android:layout_height="wrap_content"		
13	android:layout_gravity="start"		
14	android:paddingBottom="16dp"		
15	android:text="@string/app_name"		
16			
17	android:textAppearance="?android:attr/textAppearanceLarge"		/>
18			
19	<EditText		
20	android:id="@+id/costOfService"		

```

21         android:layout_width="match_parent"
22         android:layout_height="wrap_content"
23         android:hint="@string/hint_service_amount"
24         android:inputType="numberDecimal"
25         android:imeOptions="actionDone"
26         android:padding="12dp" />
27
28     <TextView
29         android:id="@+id/selectTipLabel"
30         android:layout_width="wrap_content"
31         android:layout_height="wrap_content"
32         android:text="@string/tip_percentage"
33         android:layout_marginTop="16dp"
34         android:layout_marginBottom="8dp" />
35
36     <Spinner
37         android:id="@+id/tipOptions"
38         android:layout_width="match_parent"
39         android:layout_height="wrap_content"
40         android:minHeight="48dp"
41         android:spinnerMode="dropdown" />
42
43     <androidx.constraintlayout.widget.ConstraintLayout
44         android:layout_width="match_parent"
45         android:layout_height="wrap_content"
46         android:layout_marginTop="16dp"
47         xmlns:app="http://schemas.android.com/apk/res-auto">
48
49         <TextView
50             android:id="@+id/labelText"
51             android:layout_width="wrap_content"
52             android:layout_height="wrap_content"
53             android:text="@string/round_up_tip"
54             app:layout_constraintStart_toStartOf="parent"
55             app:layout_constraintTop_toTopOf="parent"
56             app:layout_constraintBottom_toBottomOf="parent" />
57
58         <androidx.appcompat.widget.SwitchCompat
59             android:id="@+id/roundUpSwitch"
60             android:layout_width="wrap_content"
61             android:layout_height="wrap_content"
62             android:contentDescription="@string/round_up_tip"
63             app:layout_constraintEnd_toEndOf="parent"
64             app:layout_constraintTop_toTopOf="parent"
65             app:layout_constraintBottom_toBottomOf="parent" />
66
67     </androidx.constraintlayout.widget.ConstraintLayout>
68
69     <TextView
70         android:id="@+id/tipResult"
71         android:layout_width="match_parent"
72         android:layout_height="wrap_content"

```


73	android:layout_marginTop="32dp"	
74	android:text="@string/tip_result_placeholder"	
75	android:textColor="#000000"	
76	android:textSize="37sp"	
	android:textStyle="bold"	/>
	</LinearLayout>	

MainActivity.kt :

Tabel 2. Source Code Jawaban Soal 1 XML

1	package com.example.tipcalculatorxml
2	
3	import android.os.Bundle
4	import android.view.View
5	import android.widget.*
6	import androidx.appcompat.app.AppCompatActivity
7	import androidx.appcompat.widget.SwitchCompat
8	import java.text.NumberFormat
9	import kotlin.math.ceil
10	
11	class MainActivity : AppCompatActivity() {
12	
13	lateinit var costEditText: EditText
14	lateinit var tipSpinner: Spinner
15	lateinit var roundUpSwitch: SwitchCompat
16	lateinit var tipResultTextView: TextView
17	
18	override fun onCreate(savedInstanceState: Bundle?) {
19	super.onCreate(savedInstanceState)
20	setContentView(R.layout.activity_main)
21	
22	costEditText = findViewById(R.id.costOfService)
23	tipSpinner = findViewById(R.id.tipOptions)
24	roundUpSwitch = findViewById(R.id.roundUpSwitch)
25	tipResultTextView = findViewById(R.id.tipResult)
26	
27	val tipOptions = listOf("15%", "18%", "20%")
28	ArrayAdapter(
29	this,
30	android.R.layout.simple_spinner_item,
31	tipOptions
32	).also {
33	
34	it.setDropDownViewResource(android.R.layout.simple_spinner_drop
35	down_item)
36	tipSpinner.adapter = it
37	}
38	
39	costEditText.setOnEditorActionListener { _, _, ->

```

40         calculateTip()
41         false
42     }
43
44     tipSpinner.onItemSelectedListener = object :
45     AdapterView.OnItemSelectedListener {
46         override fun onItemSelected(
47             parent: AdapterView<*>, view: View?, position:
48             Int, id: Long
49         ) {
50             calculateTip()
51         }
52
53         override fun onNothingSelected(parent:
54             AdapterView<*>) {}
55     }
56
57     roundUpSwitch.setOnCheckedChangeListener { _, _ ->
58         calculateTip()
59     }
60 }
61
62 private fun calculateTip() {
63     val cost =
64     costEditText.text.toString().toDoubleOrNull()
65
66     if (cost == null) {
67
68         tipResultTextView.setText(R.string.tip_result_placeholder)
69         return
70     }
71
72     val tipPercentage = when
73     (tipSpinner.selectedItem.toString()) {
74         "20%" -> 0.20
75         "18%" -> 0.18
76         else -> 0.15
77     }
78
79     var tip = cost * tipPercentage
80     if (roundUpSwitch.isChecked) {
81         tip = ceil(tip)
82     }
83
84     val formattedTip =
85     NumberFormat.getCurrencyInstance().format(tip)
86     tipResultTextView.text = getString(R.string.tip_result,
87     formattedTip)
88 }

```

strings.xml :

Tabel 3. Source Code Jawaban Soal 1 XML

1	<resources>
2	<string name="app_name">Calculate Tip</string>
3	<string name="hint_service_amount">Bill Amount</string>
4	<string name="tip_percentage">Tip percentage</string>
5	<string name="round_up_tip">Round up tip?</string>
6	<string name="tip_result">Tip Amount: %1\$s</string>
7	<string name="tip_result_placeholder">Tip Amount: </string>
8	</resources>

Jetpack Compose :

MainActivity.kt :

Tabel 4. Source Code Jawaban Soal 1 Jetpack Compose

1	package com.example.tipcalculatorcompose
2	
3	import android.os.Bundle
4	import androidx.activity.ComponentActivity
5	import androidx.activity.compose.setContent
6	import androidx.compose.foundation.layout.*
7	import androidx.compose.foundation.text.KeyboardOptions
8	import androidx.compose.material3.*
9	import androidx.compose.runtime.*
10	import androidx.compose.ui.Alignment
11	import androidx.compose.ui.Modifier
12	import androidx.compose.ui.text.input.KeyboardType
13	import androidx.compose.ui.tooling.preview.Preview
14	import androidx.compose.ui.unit.dp
15	import java.text.NumberFormat
16	import kotlin.math.ceil
17	
18	class MainActivity : ComponentActivity() {
19	override fun onCreate(savedInstanceState: Bundle?) {
20	super.onCreate(savedInstanceState)
21	setContent { TipCalculatorApp() }
22	}
23	}
24	
25	@Composable
26	fun TipCalculatorApp() {
27	Surface(modifier = Modifier.fillMaxSize()) {
28	TipCalculatorLayout()
29	}
30	}
31	
32	@Composable
33	fun TipCalculatorLayout() {

```

34     var serviceAmount by remember { mutableStateOf("") }
35     var selectedTip by remember { mutableStateOf("20%") }
36     var roundUp by remember { mutableStateOf(false) }
37
38     val tipOptions = listOf("20%", "18%", "15%")
39     val tipPercent = selectedTip.dropLast(1).toDoubleOrNull()
40     ?: 0.0
41     val amount = serviceAmount.toDoubleOrNull() ?: 0.0
42     val formattedTip =
43     NumberFormat.getCurrencyInstance().format(
44         calculateTip(amount, tipPercent, roundUp)
45     )
46
47     Column(
48         modifier = Modifier
49             .padding(16.dp)
50             .fillMaxWidth()
51     ) {
52         with(MaterialTheme.typography) {
53             Text("Tip Calculator", style = headlineSmall)
54             Spacer(modifier = Modifier.height(16.dp))
55
56             OutlinedTextField(
57                 value = serviceAmount,
58                 onValueChange = { serviceAmount = it },
59                 label = { Text("Service Amount") },
60                 keyboardOptions = KeyboardOptions(keyboardType
61 = KeyboardType.Number),
62                 modifier = Modifier.fillMaxWidth()
63             )
64
65             Spacer(modifier = Modifier.height(16.dp))
66
67             TipDropdown(
68                 options = tipOptions,
69                 selected = selectedTip,
70                 onSelect = { selectedTip = it }
71             )
72
73             Spacer(modifier = Modifier.height(16.dp))
74
75             Row(
76                 modifier = Modifier.fillMaxWidth(),
77                 verticalAlignment = Alignment.CenterVertically
78             ) {
79                 Text("Round up tip?")
80                 Spacer(modifier = Modifier.weight(1f))
81                 Switch(checked = roundUp, onCheckedChange = {
82 roundUp = it })
83             }
84
85             Spacer(modifier = Modifier.height(24.dp))

```

```

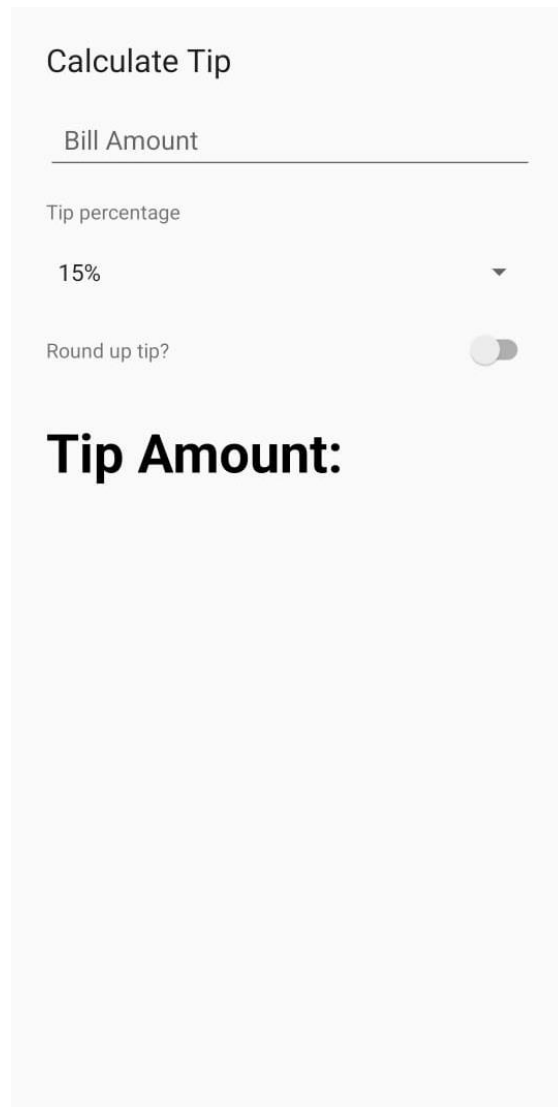
86         Text("Tip Amount: $formattedTip", style =
87         bodyLarge)
88     }
89 }
90 }
91
92 @OptIn(ExperimentalMaterial3Api::class)
93 @Composable
94 fun TipDropdown(
95     options: List<String>,
96     selected: String,
97     onSelect: (String) -> Unit
98 ) {
99     var expanded by remember { mutableStateOf(false) }
100
101     ExposedDropdownMenuBox(
102         expanded = expanded,
103         onExpandedChange = { expanded = !expanded },
104         modifier = Modifier.fillMaxWidth()
105     ) {
106         OutlinedTextField(
107             value = selected,
108             onValueChange = {},
109             readOnly = true,
110             label = { Text("Tip Percentage") },
111             trailingIcon = {
112                 ExposedDropdownMenuDefaults.TrailingIcon(expanded) },
113             modifier = Modifier
114                 .menuAnchor()
115                 .fillMaxWidth()
116         )
117         ExposedDropdownMenu(
118             expanded = expanded,
119             onDismissRequest = { expanded = false },
120             modifier = Modifier.fillMaxWidth()
121         ) {
122             options.forEach {
123                 DropdownMenuItem(
124                     text = { Text(it) },
125                     onClick = {
126                         onSelect(it)
127                         expanded = false
128                     }
129                 )
130             }
131         }
132     }
133 }
134
135 fun calculateTip(amount: Double, tipPercent: Double, roundUp:
136 Boolean): Double {
137     var tip = amount * (tipPercent / 100)

```

138	if (roundUp) tip = <i>ceil</i> (tip)
139	return tip
	}
	@Preview(showBackground = true)
	@Composable
	fun TipCalculatorPreview() {
	TipCalculatorApp()
	}

B. Output Program

XML :



Gambar 3. Screenshot Hasil Jawaban Soal 1 XML

Calculate Tip

60

Tip percentage

15%

18%

20%

Tip Amount:

Gambar 4. Screenshot Hasil Jawaban Soal 1 XML

Calculate Tip

60

Tip percentage

18%



Round up tip?



Tip Amount:
Rp11,00

Gambar 5. Screenshot Hasil Jawaban Soal 1 XML

Compose :

Tip Calculator

Service Amount

Tip Percentage

20%

Round up tip?

☐

Tip Amount: Rp0,00

Gambar 6. Screenshot Hasil Jawaban Soal 1 Jetpack Compose

Tip Calculator

Service Amount

Tip Percentage

20%

18%

15%

Gambar 7. Screenshot Hasil Jawaban Soal 1 Jetpack Compose

Tip Calculator

Service Amount

60

Tip Percentage

18%

Round up tip?

Tip Amount: Rp11,00

Gambar 8. Screenshot Hasil Jawaban Soal 1 Jetpack Compose

C. Pembahasan

XML :

activity_main.xml :

Line 1, adalah deklarasi XML standar, menyatakan bahwa file ini menggunakan XML versi 1.0 dan encoding UTF-8.

Line - sampai -, Linear layout mengatur semua komponen UI di dalamnya dengan tipe vertical, yang berarti semua elemen di dalamnya akan disusun dari atas ke bawah dengan padding 24dp(jarak antara tepi layar dan konten) agar tidak konten tidak terlalu menempel ke layar. Tinggi dan lebar layar menyesuaikan dengan parent (mengisi seluruh layar)

Line – sampai -, untuk menampilkan teks dari "@string/app_name" yang berisikan "Calculate Tip" dengan lebar dan tinggi menyesuaikan isi teks (wrap_content), posisi teksnya di kiri dengan menggunakan layout_gravity="start", dengan jarak bawah 16dp, dengan tampilan teks besar (textAppearanceLarge)

Line – sampai -, adalah kolom untuk meng-input teks, input berupa angka desimal (numberDecimal), dan keyboard akan menampilkan tombol "Done"

Line 26 sampai 32, Menampilkan teks "Tip Percentage" dengan lebar dan tinggi menyesuaikan dengan isi teks

Line 34 sampai 39, menampilkan spinner dengan mode dropdown

Line 41 sampai 45, membuat layout yang fleksibel dengan constraint antar komponen dari line 41 sampai 65

Line 47 sampai 54, menampilkan teks "Round up tip?" dengan constraint awalan dari parent (kiri) dan constraint atas dan bawah dari parent

Line 56 sampai 63, menampilkan sebuah switch dengan constraint akhiran dari parent (kanan) dan constraint atas dan bawah dari parent

Line 67 sampai 76, menampilkan teks "Tip Amount" dengan ukuran 37sp, warna hitam, dan style bold

MainActivity.kt :

Line 1, menyatakan bahwa file ini berada di dalam package com.example.tipcalculatorxml

Line 3 sampai 9, Mengimpor semua class dan fungsi yang dibutuhkan:

Bundle: untuk menyimpan dan memulihkan state aktivitas.

View, EditText, Spinner, TextView, AdapterView: komponen UI.

AppCompatActivity: base class untuk activity dengan dukungan ActionBar.

SwitchCompat: komponen switch modern dari AndroidX.

NumberFormat: untuk format mata uang.

ceil: membulatkan angka ke atas.

Line 11, Mendefinisikan MainActivity sebagai subclass dari AppCompatActivity.

Line 13 sampai 16, mendefinisikan variabel untuk menyimpan referensi ke elemen UI di layout (EditText, Spinner, SwitchCompat, dan TextView).

Line 18 sampai 20, adalah fungsi yang dipanggil saat activity pertama kali dibuat.

setContentview() menghubungkan file XML activity_main.xml dengan kelas Kotlin ini.

Line 22 sampai 25, Menghubungkan masing-masing variabel dengan elemen UI yang berada di layout activity_main berdasarkan id.

Line 27, mendeklarasikan variabel "TipOptions" untuk menyimpan daftar pilihan tip yang bersifat immutable (tidak bisa dirubah) karena menggunakan fungsi listOf

Line 28 sampai 35, Membuat ArrayAdapter untuk menghubungkan tipOptions ke Spinner, menentukan tampilan dropdown, dan mengatur adapter ke tipSpinner.

Line 37 sampai 40, Ketika pengguna menekan enter/selesai setelah mengisi EditText, fungsi calculateTip() akan dipanggil.

Line 42 sampai 50, Ketika pengguna memilih item dari spinner, akan memanggil calculateTip().

Line 52 sampai 54, Ketika switch diubah (on/off), akan memicu kalkulasi ulang.

Line 57 sampai 58, Mengambil teks dari EditText, mengubah ke Double. Kalau kosong atau bukan angka, hasilnya null.

Line 60 sampai 63, Jika input tidak valid, tampilkan placeholder dan keluar dari fungsi.

Line 65 sampai 69, Menentukan persentase tip sesuai pilihan dari spinner.

Line 71 sampai 74, Mengalikan cost dengan tipPercentage. Jika switch aktif, hasilnya dibulatkan ke atas.

Line 76 dan 77, Format hasil ke dalam mata uang lokal (misal: Rp jika device disetel ke Indonesia dan \$ jika device disetel ke US) lalu ditampilkan ke TextView.

Jetpack Compose :

MainActivity.kt:

Line 1, Menyatakan bahwa file ini berada dalam package com.example.tipcalculatorcompose.

Line 3 sampai 16, Mengimpor semua komponen yang dibutuhkan, termasuk:

ComponentActivity, setContent() untuk activity Compose.

Modifier, Column, Text, OutlinedTextField, Switch, Dropdown, dll dari Compose.

remember, mutableStateOf untuk state management.

NumberFormat dan ceil untuk menghitung dan memformat tip.

Line 18 sampai 23, MainActivity adalah entry point dari aplikasi. setContent { TipCalculatorApp() } akan menampilkan UI dari fungsi TipCalculatorApp().

Line 25 sampai 30, Fungsi Compose utama yang membungkus UI dalam sebuah Surface, fillMaxSize() membuatnya memenuhi seluruh layar, lalu memanggil fungsi TipCalculatorLayout() untuk isi layoutnya.

Line 34, Menyimpan input pengguna untuk jumlah uang (EditText).

Line 35, Menyimpan pilihan tip (%) dengan default 20%.

Line 36, Menyimpan status switch untuk membulatkan tip atau tidak dengan default off (false).

Line 38, Daftar opsi dropdown tip.

Line 39, Mengambil angka dari selectedTip (misal "20%") menjadi 20.0.

Line 40, Mengubah input pengguna ke Double, default ke 0.0 jika kosong/invalid.

Line 41 dan 43, Memanggil calculateTip dan memformat hasilnya menjadi bentuk uang.

Line 45 sampai 49, Mengatur layout vertikal dengan padding 16dp dan lebar penuh.

Line 51, Menampilkan teks judul aplikasi yaitu "Tip Calculator".

Line 52, spacer untuk memberikan jarak sebesar 16dp

Line 54 sampai 60, Input untuk jumlah layanan (serviceAmount), hanya menerima angka (keyboardType = Number).

Line 64 sampai 68, Dropdown untuk memilih persentase tip.

Line 72 sampai 79, Baris dengan teks dan switch untuk membulatkan tip.

Line 82, Menampilkan hasil tip dalam bentuk mata uang.

Line 87 sampai 93, Fungsi dropdown custom dengan ExposedDropdownMenuBox dari Material 3.

Line 94, Menyimpan status apakah dropdown sedang terbuka atau tidak.

Line 96 sampai 100, Komponen dari Material 3 untuk dropdown menu terintegrasi dengan OutlinedTextField.

Line 101 sampai 110, Text field yang menampilkan pilihan saat ini (tidak bisa diketik/manual).

Line 111 sampai 125, daftar opsi yang muncul ketika dropdown dibuka.

Ketika item diklik: Memanggil onSelect() untuk mengganti nilai dan menutup dropdown (expanded = false).

Line 129 sampai 133, Menghitung tip dari amount * persen. Jika roundUp == true, hasilnya dibulatkan ke atas dengan ceil().

Line 135 sampai 139, Untuk menampilkan preview UI di Android Studio tanpa menjalankan app-nya.

D. Tautan Git

Berikut adalah tautan untuk source code yang telah dibuat.

<https://github.com/NazmiHakim/Pemrograman-Mobile/tree/main/Modul2>

SOAL 2

Jelaskan perbedaan dari implementasi XML dan Jetpack Compose beserta kelebihan dan kekurangan dari masing-masing implementasi.

A. Pembahasan

Perbedaan implementasi XML dan Jetpack Compose terletak pada pendekatan dan cara mendesain tampilan. XML menggunakan deklarasi UI secara terpisah dalam file layout .xml, sedangkan Jetpack Compose menggunakan kode Kotlin langsung untuk mendefinisikan UI secara deklaratif.

Di XML, UI didefinisikan secara statis dalam file XML, lalu dikaitkan dengan logika di file Kotlin/Java menggunakan findViewById() atau view binding. Kelebihannya adalah banyak developer sudah familiar, banyak referensi dan dokumentasi yang tersedia, serta mudah diintegrasikan dengan berbagai library UI lama. Kekurangannya, proses pengembangan terasa lebih terpisah antara UI dan logika, membutuhkan lebih banyak boilerplate code, dan cenderung kurang fleksibel saat UI kompleks atau dinamis.

Di Jetpack Compose, UI dibangun secara deklaratif di dalam kode Kotlin, tanpa perlu file XML terpisah. Kelebihannya, kode menjadi lebih ringkas, lebih mudah dibaca, dan lebih fleksibel. Compose mendukung state management secara langsung sehingga lebih mudah untuk membuat UI dinamis dan interaktif. Kekurangannya, masih terbilang baru sehingga dokumentasi atau dukungan komunitas belum sebanyak XML, dan belum semua fitur Android mendukung Compose secara penuh (terutama untuk project lama).